

ES98A Assignment 1

Author: Jabir Hussain (2007942)

NB: Across the assignment, I redefine the same variables (i.e. `J = 10_000`). This is done to ensure that variables match their definition at the start of the exercise rather than relying on variables declared at the top of the notebook. Whilst I recognise the memory inefficiency this comes with, it is done for the sake of readability and reliability.

```
In [12]: import numpy as np
import matplotlib.pyplot as plt

np.random.seed(2007942)
```

The code cell below contains the Runge-Kutta method provided by the assignment brief.

```
In [13]: def RK4A(f, T, x0):
    # Fourth-order Runge-Kutta solver for the autonomous ODE system
    # \dot{x} = f(x) with initial condition x0 at time T[0].
    # T is required to be a one-dimensional numpy array of times at which
    # the solution is desired; the initial condition x0 is required to be a
    # numpy array with shape (n,) for some positive integer n; and f is a
    # function accepting a numpy shape (n,) array and returning another
    # numpy shape (n,) array.
    # RK4A returns the time array T and an array X with shape (len(T), n),
    # where X[i,:] gives the solution value at time T[i].

    assert len(T.shape) == 1
    assert len(x0.shape) == 1
    assert x0.shape == f(x0).shape

    X = np.zeros((len(T), len(x0)))
    X[0,:] = x0

    for i in range(1, len(T)):
        dt = T[i] - T[i-1]
        xp = X[i-1,:]
        k1 = f(xp)
        k2 = f(xp + 0.5 * dt * k1)
        k3 = f(xp + 0.5 * dt * k2)
        k4 = f(xp + dt * k3)
        X[i,:] = xp + dt * (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0
    return T, X
```

Exercise 1.1.

The SIR model is a simple first-order ODE system modelling the spread of a disease in a well-mixed population of (N) individuals.

It models the evolution of a three-dimensional state variable $x = (S, I, R)$ over time, where the components represent the numbers of individuals in the population that are susceptible to, infected with, and recovered from the disease:

$$\begin{aligned}\frac{dS}{dt} &= -\frac{\beta IS}{N}, \\ \frac{dI}{dt} &= \frac{\beta IS}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I.\end{aligned}$$

The initial condition $x(0)$ consists of three non-negative numbers with sum N , and β and γ are positive parameters.

- β represents the transmission rate, describing how frequently susceptible and infected groups interact with one another to mark new cases.
- γ represents the recovery rate for infected cases.
- N is the total population; in an SIR model, this remains constant as death is not accounted for.

```
In [14]: beta, gamma, N = 2.0, 1.0, 1000.0
initial_state = np.array([999.0, 1.0, 0.0]) # x(0) = (999, 1, 0)
time_grid = np.linspace(0.0, 20.0, int(20.0 / 0.25) + 1)
```

We define a function `sir_derivatives` that computes each value for $\frac{dS}{dt}$, $\frac{dI}{dt}$, and $\frac{dR}{dt}$ from the formulae above.

```
In [15]: def sir_derivatives(state: np.ndarray, beta: float, gamma: float, population: float) -> np.ndarray:
        """
        Compute derivatives (dS/dt, dI/dt, dR/dt) for the SIR model.
        Returns derivative vector as a numpy array.
        Type hinting used for sanity checks for parameter types.
        """
        assert beta > 0 and gamma > 0 and population > 0, "β, γ, N must be positive."
        assert state.shape == (3,), "State vector must have length 3."
        S, I, R = state
        assert np.all(state >= 0), "Populations must remain non-negative."
        dS_dt = -beta * S * I / population
        dI_dt = beta * S * I / population - gamma * I
        dR_dt = gamma * I
        return np.array([dS_dt, dI_dt, dR_dt])
```

In order to perform the integration with the provided `RK4A` method, we define a lambda function `rhs`. `RK4A` expects a single function `f` as an argument whereas the `sir_derivatives` method has 4 arguments. A direct call of `sir_derivatives` would yield a `TypeError` for the missing arguments, and given that the `RK4A` template must remain unchanged, a lambda function is used as a Python closure to map the argument to the `sir_derivatives` method.

The Runge-Kutta method is used to numerically integrated the system of equations. As described in the `RK4A` docstring, it returns both the time array `T` and the solution at time `T=t`.

```
In [16]: # integrate
rhs = lambda x: sir_derivatives(x, beta, gamma, N)
T, trajectory = RK4A(rhs, time_grid, initial_state)
susceptible, infected, recovered = trajectory.T
```

The population values for S , I and R are indexed from their respective subarrays. In addition, the derivative values are also extracted: if the derivative values are small then we can determine whether the system of equations have reached a steady state.

```
In [17]: t_final = 20.0
idx = int(np.argmax(np.abs(T - t_final))) # find index for t=20.0

S_20, I_20, R_20 = susceptible[idx], infected[idx], recovered[idx] # function values at t=20.0
dS_20, dI_20, dR_20 = sir_derivatives(trajectory[idx], beta, gamma, N) # derivative values to determine

plt.figure(figsize=(8,5))
plt.plot(T, susceptible, label=r"$S(t)$")
plt.plot(T, infected, label=r"$I(t)$")
plt.plot(T, recovered, label=r"$R(t)$")

plt.scatter([T[idx]]*3, [S_20, I_20, R_20], zorder=3)
plt.axvline(T[idx], ls="--", lw=1, color="gray", alpha=0.6)

# mark final values
plt.text(T[idx]+1.5, S_20, f"S(20)={S_20:.2f}")
plt.text(T[idx]+1.5, I_20, f"I(20)={I_20:.2f}")
plt.text(T[idx]+1.5, R_20, f"R(20)={R_20:.2f}")
fig1="Figure 1. SIR trajectories integrated using the provided RK4A function, with parameters x_0 = (999
plt.figtext(0.5, -0.05, fig1, wrap=True, ha='center', fontsize=12)

plt.xlabel(r"Time $t$ (model units)")
plt.ylabel("Population count")
plt.title("2007942 - SIR trajectories solved with Runge-Kutta integration")
plt.legend()
plt.tight_layout()
plt.show()

print(f"\nAt t = {T[idx]:.2f}: S(20) = {S_20:.2f}, I(20) = {I_20:.2f}, R(20) = {R_20:.2f}")
print(f"Rates at t=20: |dS/dt| = {abs(dS_20):.4e}, |dI/dt| = {abs(dI_20):.4e}, |dR/dt| = {abs(dR_20):.4e}")
```

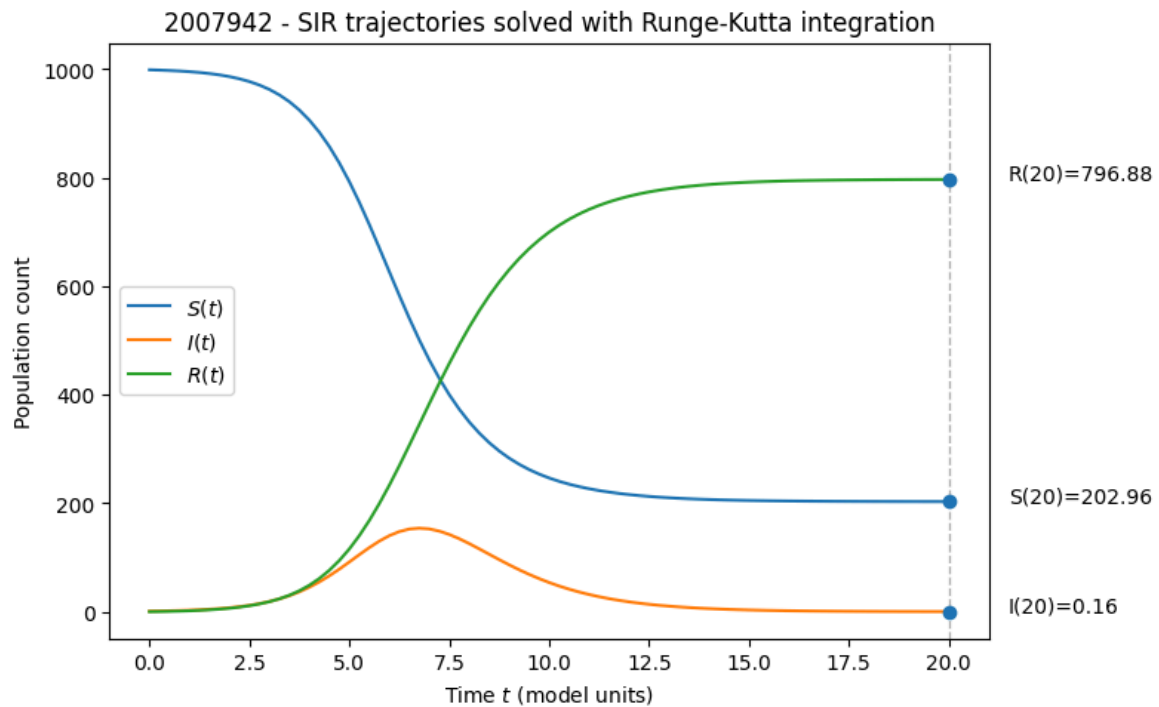


Figure 1. SIR trajectories integrated using the provided RK4A function, with parameters $x_0 = (999, 1, 0)$, $\beta = 2.0$, $\gamma = 1.0$

At $t = 20.00$: $S(20) = 202.96$, $I(20) = 0.16$, $R(20) = 796.88$
 Rates at $t=20$: $|dS/dt| = 6.6191e-02$, $|dI/dt| = 9.6876e-02$, $|dR/dt| = 1.6307e-01$

The graph highlights the following behaviour for each group of individuals:

- The susceptible group, $S(t)$, starts with 999 people. The curve decreases monotonically as people become more infected. The curve plateaus, with a final value of $S(20) = 202.96$
- The infected group, $I(t)$, starts with 1 person. The curve rises and peaks at the point where the number of new infections is equal to the number of recoveries. The plot shows that $I(20) = 0.16$.
- The recovered group, $R(t)$, starts at 0, increasing monotonically as people leave the infected group. It plateaus at $R(20) = 796.88$.

It should be noted that at $t = 20.0$, the system shows fractional people. The SIR model is a deterministic, continuous set of equations rather than presenting discrete counts. In addition, the use of a numerical integrator such as the Runge-Kutta method computes the solutions with floating point arithmetic, approximating the continuous evolution of the system. These values should be viewed as expected averages at time t rather than a fractional person.

The derivatives at $t = 20$ are shown to have small magnitudes, highlighting that the system is essentially stationary at this time.

Exercise 1.2.

In this exercise, we assume β follows the uniform distribution $\beta \sim U([1.5, 2.5])$. There are 10^4 independent samples drawn from the distribution and for each sampled β_j , we integrate the same SIR system from Exercise 1.1, with initial state $x_0 = (999, 1, 0)$ and parameters $\gamma = 1.0$, $N = 1000$.

```
In [18]: J = 10_000 # samples
         beta_samples = np.random.uniform(1.5, 2.5, size=J)

         n_instances = 500

         gamma, N = 1.0, 1000.0
         initial_state = np.array([999.0, 1.0, 0.0])
         time_grid = np.linspace(0.0, 20.0, int(20.0 / 0.25) + 1)
```

A total of 500 random instances are drawn, stored in their respective arrays. As we iterate across said samples, we use **RK4A** to integrate the S , I and R derivatives. They are plotted on separate axes. The bold line highlights the curve for $\beta = 2.0$, with dotted lines highlighting the curves for the minimum and maximum values of β .

```
In [19]: # defining numpy arrays for the results of each trajectory
         overlay_S = np.empty((n_instances, len(time_grid)))
         overlay_I = np.empty((n_instances, len(time_grid)))
         overlay_R = np.empty((n_instances, len(time_grid)))
```

```

for j, beta_j in enumerate(beta_samples[:n_instances]):
    # integrate
    rhs = lambda x: sir_derivatives(x, beta_j, gamma, N)
    _, X = RK4A(rhs, time_grid, initial_state)
    overlay_S[j], overlay_I[j], overlay_R[j] = X.T

# make subplots
fig, ax = plt.subplots(3, 1, figsize=(8, 10), sharex=True)

for k in range(n_instances):
    ax[0].plot(time_grid, overlay_S[k], color="C0", alpha=0.05)
    ax[1].plot(time_grid, overlay_I[k], color="C1", alpha=0.05)
    ax[2].plot(time_grid, overlay_R[k], color="C2", alpha=0.05)

# make beta = 2.0 bold on plot
ax[0].plot(T, susceptible, color="C0", lw=2.0, label="Baseline  $\beta=2.0$ ")
ax[1].plot(T, infected, color="C1", lw=2.0)
ax[2].plot(T, recovered, color="C2", lw=2.0)

beta_min, beta_max = 1.5, 2.5
# minimum beta (= 1.5)
rhs_min = lambda x: sir_derivatives(x, beta_min, gamma, N)
_, Xmin = RK4A(rhs_min, time_grid, initial_state)

# maximum beta (= 2.5)
rhs_max = lambda x: sir_derivatives(x, beta_max, gamma, N)
_, Xmax = RK4A(rhs_max, time_grid, initial_state)

# plot curves
ax[0].plot(time_grid, Xmin[:,0], '--', color='blue', lw=1.5, label=r'$\beta_{\min}=1.5$')
ax[1].plot(time_grid, Xmin[:,1], '--', color='blue', lw=1.5)
ax[2].plot(time_grid, Xmin[:,2], '--', color='blue', lw=1.5)

ax[0].plot(time_grid, Xmax[:,0], '--', color='red', lw=1.5, label=r'$\beta_{\max}=2.5$')
ax[1].plot(time_grid, Xmax[:,1], '--', color='red', lw=1.5)
ax[2].plot(time_grid, Xmax[:,2], '--', color='red', lw=1.5)

ax[0].legend(loc="upper right")

ax[2].set_xlabel(r"Time $t$ (model units)")
ax[0].set_ylabel(r"$S(t)$ (susceptible population)")
ax[1].set_ylabel(r"$I(t)$ (infected population)")
ax[2].set_ylabel(r"$R(t)$ (recovered population)")

fig.suptitle(r"2007942 - Dispersion of SIR trajectories under uncertain  $\beta \sim U([1.5, 2.5])$ ")
fig2 = "Figure 2\nDispersion of SIR trajectories under uncertain  $\beta \sim U([1.5, 2.5])$ , showing 500 random re
plt.figtext(0.5, -0.1, fig2, wrap=True, ha='center', fontsize=12)
ax[0].legend(loc="upper right")
plt.tight_layout()
plt.show()

```

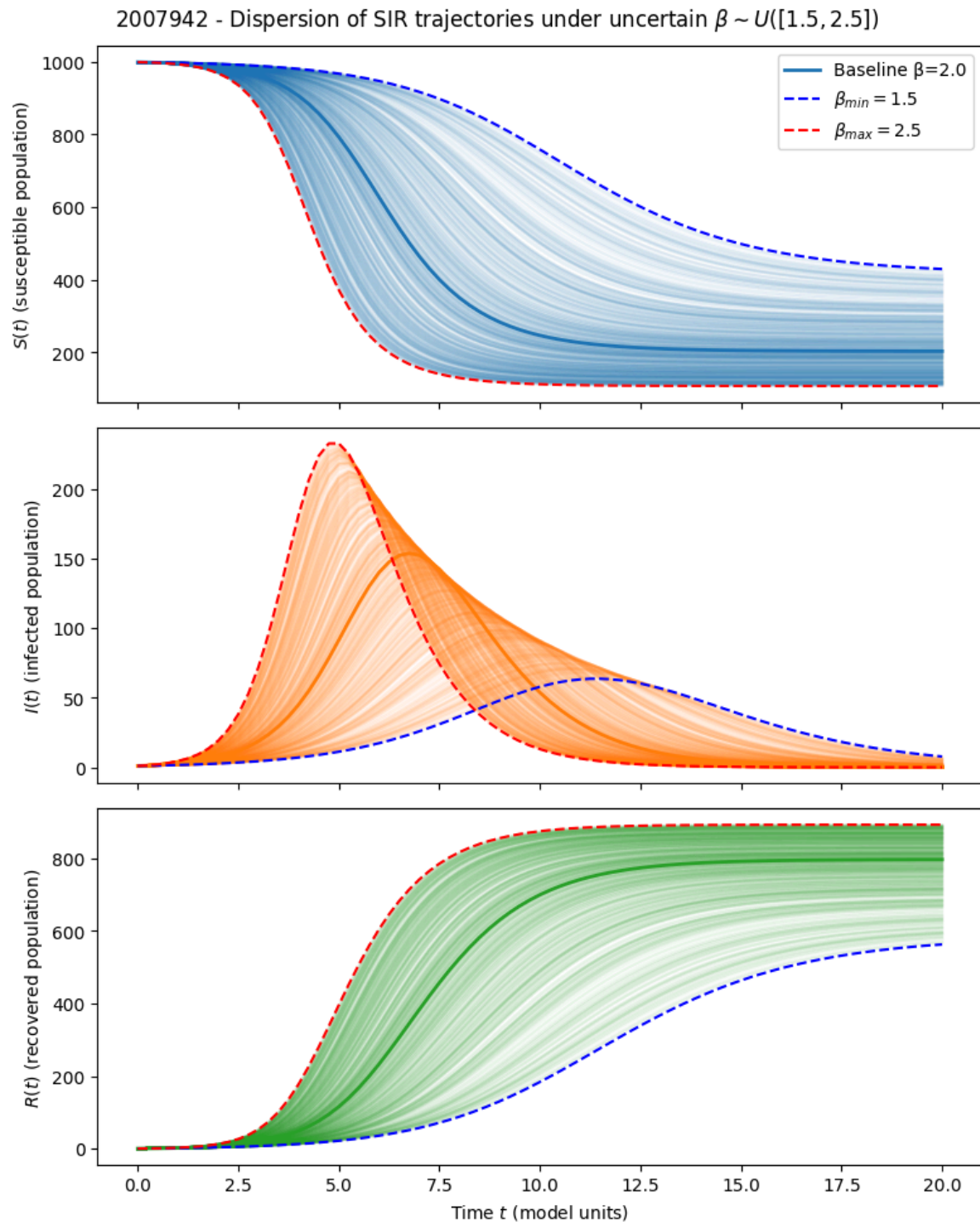


Figure 2

Dispersion of SIR trajectories under uncertain $\beta \sim U([1.5, 2.5])$, showing 500 random realisations. The bold line denotes the baseline case $\beta = 2.0$. The dashed lines represent the curves for the minimum and maximum values of β .

Figure 2 shows that at early stages of the model ($t \lesssim 3$), all three plots are close in value. This suggests that the system's behaviour at the beginning is insensitive to the variations of β . As t grows, the graph shows the spread of realisations are distinct.

The $S(t)$ plot shows that as β grows, so does the depletion of susceptibles. All the curves for $S(t)$ plateau by the end of the time period, demonstrating stability in the decline of the susceptible population.

The $R(t)$ plot mirror the behaviour of the $S(t)$ function. For larger values of β , the population recovers at a faster rate, with values plateauing as $t \rightarrow 20$, showing stability in the increase of recovered individuals.

The $I(t)$ plot has the greatest dispersion between the 3 plots. Small values of β show a significantly delayed and flatter peak, whereas larger values of β have an earlier and steeper rise. This visually shows that as β varies, the magnitude and timing of the infections peak have the largest variation.

```

In [20]: J = 10_000
beta_samples = np.random.uniform(1.5, 2.5, size=J)
s_20_samples = np.empty(J)

for j, beta_j in enumerate(beta_samples):
    rhs = lambda x: sir_derivatives(x, beta_j, gamma, N)
    _, X = RK4A(rhs, time_grid, initial_state)
    s_20_samples[j] = X[-1, 0] # store final susceptible value

# Compute statistics
s_20_mean = np.mean(s_20_samples)
s_20_std = np.std(s_20_samples)

print(f"Mean of S(20): {s_20_mean:.2f}")
print(f"Standard deviation of S(20): {s_20_std:.2f}")

plt.figure(figsize=(7,5))
plt.hist(s_20_samples, bins=50, color="C0", edgecolor='black', alpha=0.7)

# plot mean and stdev lines
plt.axvline(s_20_mean, color='k', linestyle='--', label=f"Mean = {s_20_mean:.2f}")
plt.axvline(s_20_mean + s_20_std, color='gray', linestyle=':', label=f"+1σ = {s_20_mean + s_20_std:.2f}")
plt.axvline(s_20_mean - s_20_std, color='gray', linestyle=':', label=f"-1σ = {s_20_mean - s_20_std:.2f}")

plt.xlabel(r"$S(20)$")
plt.ylabel("Frequency")
plt.title(r"2007942 - Distribution of $S(20)$ under $\beta \sim U([1.5, 2.5])$")
fig3 = r"Figure 3: A histogram that highlights the distribution of the final susceptible population $S(20)$"
plt.figtext(0.5, -0.1, fig3, wrap=True, ha='center', fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()

```

Mean of S(20): 224.09

Standard deviation of S(20): 88.76

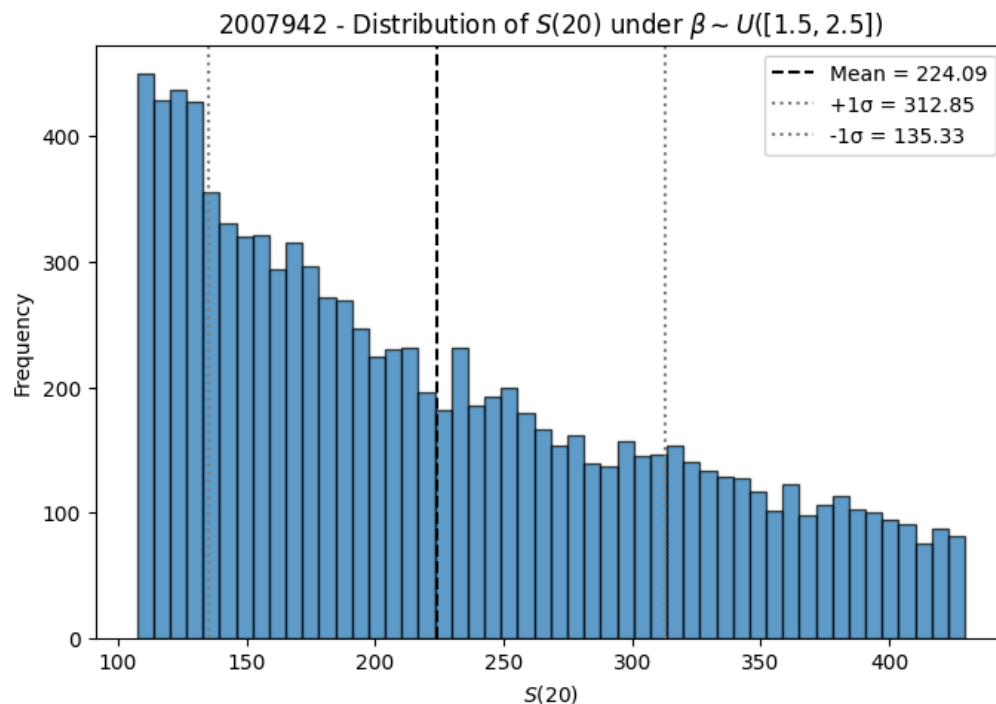


Figure 3: A histogram that highlights the distribution of the final susceptible population $S(20)$ across the 10^4 realisations of β in the uniform distribution.

The histogram for $S(20)$ highlights it is a right-skewed unimodal distribution, showing that $S(20)$ is non-linearly dependent on β . This means the majority of simulations end with a low number of susceptible individuals, in line with the faster infection rate for larger values of β . The sample mean and standard deviation are 224.09 and 88.76 respectively.

Exercise 1.3.

In this exercise, the sample average and standard deviations for $S(20)$ will be calculated across the first 10, 100, 1000 and 10^4 β samples.

```
In [21]: n_arr = np.array([10, 100, 1000, 10_000])
means = [np.mean(s_20_samples[:n]) for n in n_arr]
stdevs = [np.std(s_20_samples[:n]) for n in n_arr]

for n, m, s in zip(n_arr, means, stdevs):
    print(f"For {n} samples: mean = {m:.2f}, std = {s:.2f}")

plt.figure(figsize=(7,5))
plt.plot(n_arr, means, "o-", label="Sample mean")
plt.plot(n_arr, stdevs, "s-", label="Sample std")
plt.xscale("log")
plt.xlabel("Number of samples (log scale)")
plt.ylabel(r"Estimated value of $S(20)$ statistic")
plt.title(r"2007942 - Convergence of $\bar{S}_n$ and $s_n$ with sample size")
plt.legend()
fig4 = r"Figure 4: A graph that shows the convergence of the sample mean and standard deviation of $S(20)$"
plt.figtext(0.5, -0.1, fig4, wrap=True, ha='center', fontsize=12)
plt.grid(True, which="both", ls="--", alpha=0.5)
plt.tight_layout()
plt.show()
```

For 10 samples: mean = 231.98, std = 64.40
 For 100 samples: mean = 225.47, std = 93.75
 For 1000 samples: mean = 229.60, std = 90.23
 For 10000 samples: mean = 224.09, std = 88.76

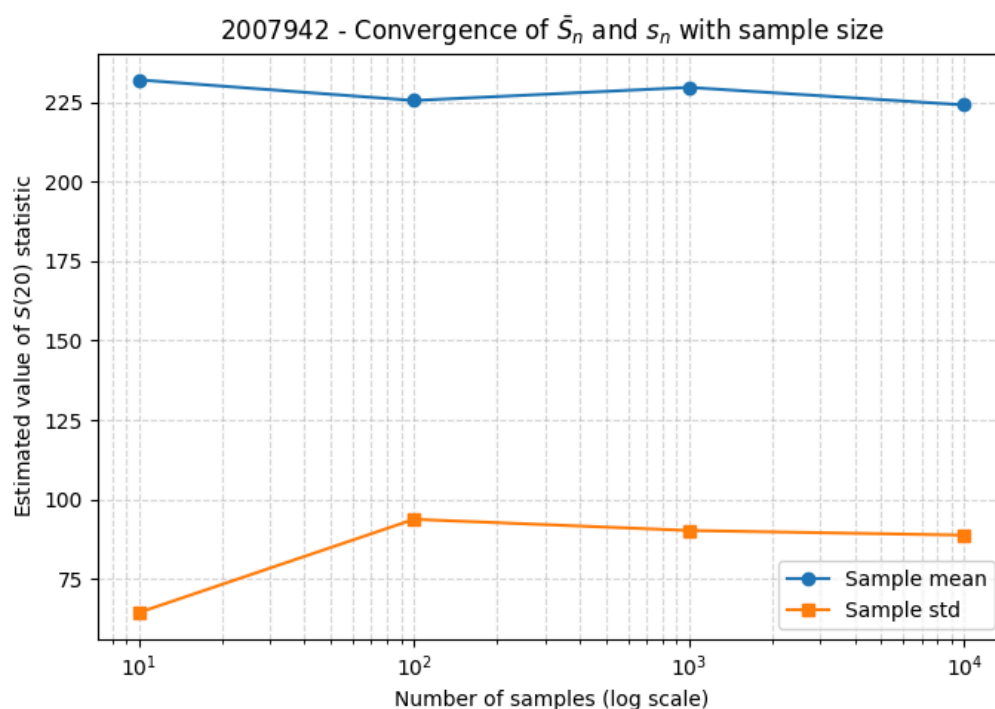


Figure 4: A graph that shows the convergence of the sample mean and standard deviation of $S(20)$ as the number of β samples increase.

There is a noticeable jump in the standard deviations between $n = 10$ samples and $n = 100$ samples, highlighting the variability at small sample sizes. As the number of β samples increase, the mean and standard deviation of $S(20)$ becomes more stable, as the random sampling noise is cancelled out. In turn, the mean and standard deviation converge toward their true values, an empirical demonstration of the Law of Large Numbers.

Exercise 1.4.

This exercise looks to find time T at which the number of infected individuals reaches its maximum whilst quantifying said maximum number of infected individuals.

```
In [22]: J = 10_000
beta_samples = np.random.uniform(1.5, 2.5, size=J)

Ts = np.empty(J)
ITs = np.empty(J)

for j, beta_j in enumerate(beta_samples):
    rhs = lambda x: sir_derivatives(x, beta_j, gamma, N)
    _, X = RK4A(rhs, time_grid, initial_state)
```



```

    I_traj = X[:, 1] # infected population over time
    T_idx = np.argmax(I_traj)
    Ts[j] = time_grid[T_idx]
    ITs[j] = I_traj[T_idx]

T_mean, T_stdev = np.mean(Ts), np.std(Ts)
IT_mean, IT_stdev = np.mean(ITs), np.std(ITs)

print(f"Mean(T) = {T_mean:.2f}, Std(T) = {T_stdev:.2f}")
print(f"Mean(I(T))= {IT_mean:.2f}, Std(I(T))= {IT_stdev:.2f}")

plt.hist(Ts, bins=50, color="C1", alpha=0.7, edgecolor='black')
plt.axvline(T_mean, color='k', ls='--', label=f"Mean = {T_mean:.2f}")
plt.xlabel(r"Peak time $T$")
plt.ylabel("Frequency")
plt.title(r"2007942 - Distribution of peak times $T$")
plt.legend()
fig5a = r"Figure 5(a): a histogram plot that shows the distribution of infection peak times T across 100
plt.figtext(0.5, -0.1, fig5a, wrap=True, ha='center', fontsize=12)
plt.tight_layout()
plt.show()

plt.figure(figsize=(7,5))
plt.hist(ITs, bins=50, color="C2", alpha=0.7, edgecolor='black')
plt.axvline(IT_mean, color='k', ls='--', label=f"Mean = {IT_mean:.2f}")
plt.xlabel(r"Peak infection $I(T)$")
plt.ylabel("Frequency")
plt.title(r"2007942 - Distribution of infection peaks $I(T)$")
plt.legend()
fig5b = r"Figure 5(b): a histogram plot that shows the distribution of the maximum infection values I(T)
plt.figtext(0.5, -0.1, fig5b, wrap=True, ha='center', fontsize=12)
plt.tight_layout()
plt.show()

plt.figure(figsize=(7,6))
plt.scatter(Ts, ITs, s=8, alpha=0.1, color="C0")
plt.scatter(T_mean, IT_mean, color="red", s=50, label="Joint mean")
plt.text(
    T_mean + 0.15, IT_mean, # slight offset so it doesn't overlap the point
    fr"($\bar{\{T\}}$={T_mean:.2f}, $\overline{\{I(T)\}}$={IT_mean:.2f})",
    fontsize=11, color="red", va="center"
)
plt.xlabel(r"Peak time $T$")
plt.ylabel(r"Peak infection $I(T)$")
plt.title(r"2007942 - Joint distribution of $T$ and $I(T)$")
plt.legend()
fig5c = r"Figure 5(c): a histogram plot that shows the joint distribution of infection peak time T and p
plt.figtext(0.5, -0.1, fig5c, wrap=True, ha='center', fontsize=12)
plt.tight_layout()
plt.show()

Mean(T) = 7.21, Std(T) = 1.80
Mean(I(T))= 151.20, Std(I(T))= 49.71

```

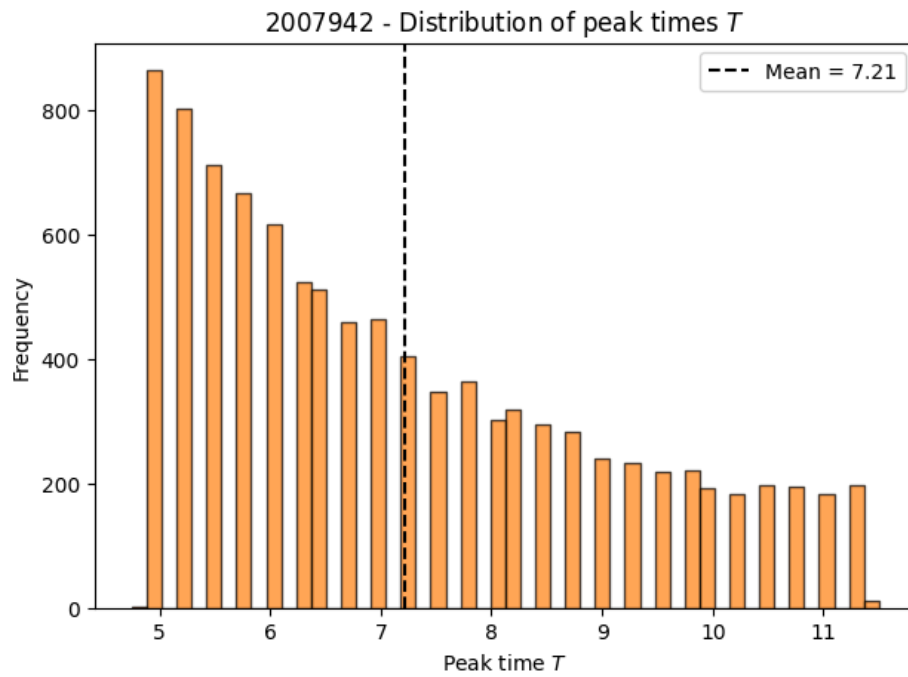



Figure 5(a): a histogram plot that shows the distribution of infection peak times T across 10000 simulations with $\beta \sim U([1.5, 2.5])$.

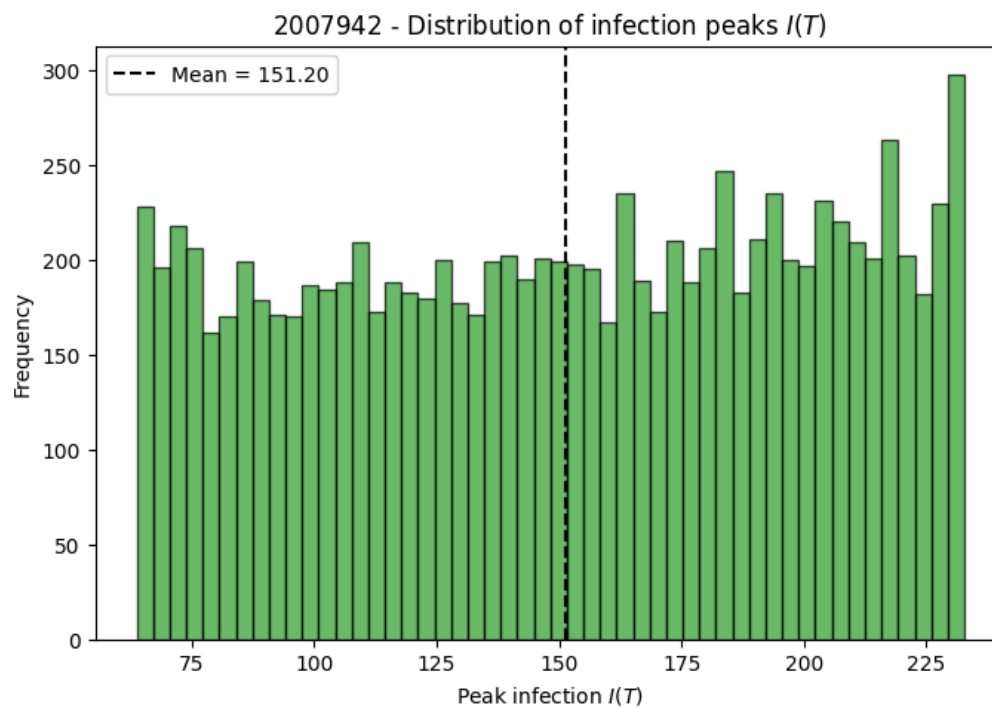


Figure 5(b): a histogram plot that shows the distribution of the maximum infection values $I(T)$ across 10000 simulations with $\beta \sim U([1.5, 2.5])$.

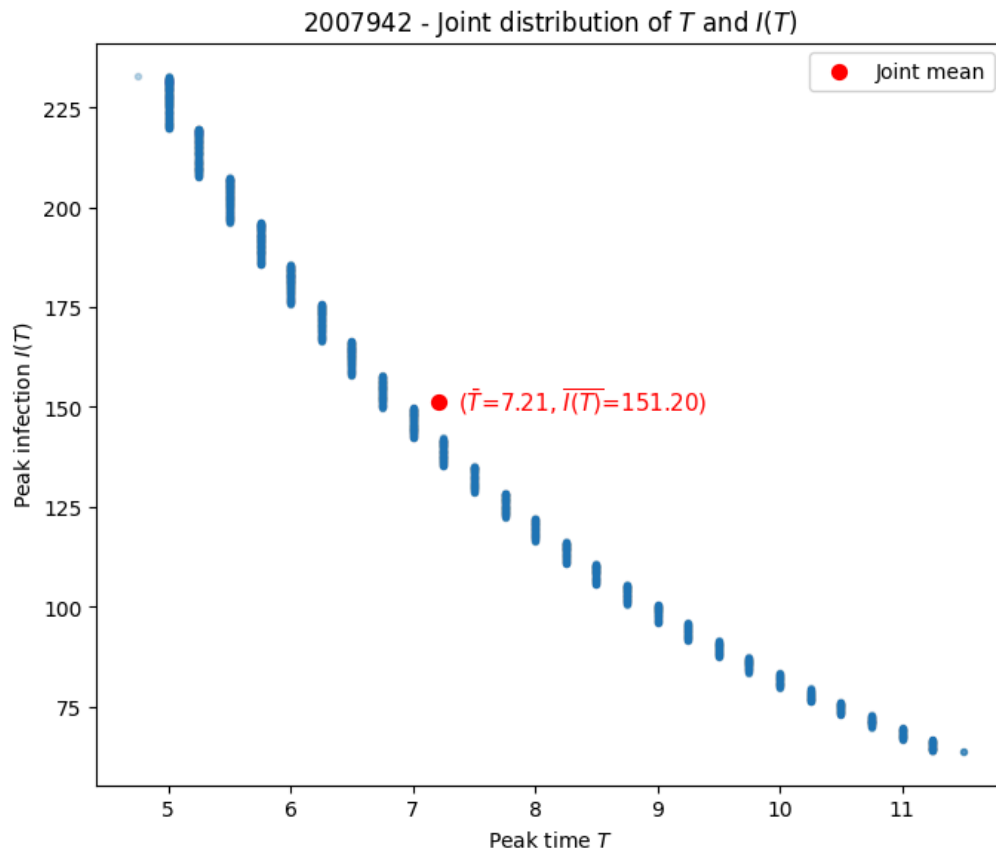


Figure 5(c): a histogram plot that shows the joint distribution of infection peak time T and peak magnitude $I(T)$ across 10000 simulations with $\beta \sim U([1.5, 2.5])$.

The first plot shows a right-skewed unimodal distribution as a histogram for peak times T with mean peak time $\bar{T} = 7.21$. The skewness shows that outbreaks with higher β values are more frequent. Higher infection rates lead to earlier epidemic peaks.

The second plot shows a near-uniform distribution for maximum infection levels $I(t)$, with a mean of 151.20. This near-uniform spread of peak magnitudes shows that the infection peaks scale linearly with values of β across the sampled range.

The third and final plot is a joint scatter plot of both T and $I(T)$. This plot is monotonically decreasing; an epidemic that peaks later has a smaller peak than those with higher levels of infection. Large values of β translate to a faster transmission rate, raising $I(T)$ and reducing T .